



Construire des agents IA fiables

Comprendre, construire et vérifier une boucle agentique en Python.

50 MINUTES

Un concept, son code, puis son comportement observable.

- | | | |
|-----------|---|---------------|
| 01 | Comprendre
Distinguer modèle, outil, agent et application. | 9 min |
| 02 | Construire
Assembler proposition, exécution et observation. | 20 min |
| 03 | Fiabiliser
Valider les preuves et contrôler les actions. | 13 min |
| 04 | Évaluer
Tester les chemins normaux et les refus attendus. | 8 min |

09:42 : une clinique signale une hausse de température.

Alerte CC-204

Clinique KCARE-ADJ-01

12,4 °C depuis 52 min

Vaccins · lot VX-204

Le réfrigérateur de la clinique a déclenché une alerte. Votre équipe doit vérifier les mesures et retrouver la procédure à suivre.

Vous êtes l'équipe de garde.

Construire un agent qui rassemble les preuves, prépare l'incident et demande la décision d'un opérateur simulé.

Exercice fictif : données, procédures et opérateur simulés. Aucun acte médical ni aucune action physique.

12,4 °C est une mesure, pas une moyenne.

Observation

Le capteur indique 12,4 °C depuis 52 minutes. Son état est OK.

Règle du lab

La plage est 2–8 °C. Au-dessus de 8 °C pendant au moins 30 minutes, le cas est CRITICAL.

Décision attendue

Consulter la procédure, créer le dossier et obtenir la revue requise avant de conclure.

Les données, la procédure et l'opérateur sont simulés. Aucune action médicale ou physique n'est exécutée.

Un modèle produit une proposition. Une application agit.

Modèle

Il reçoit des messages et prédit une réponse : du texte ou un appel structuré.

Outil

C'est une fonction écrite par le développeur : lire une donnée, appliquer une règle ou créer un dossier.

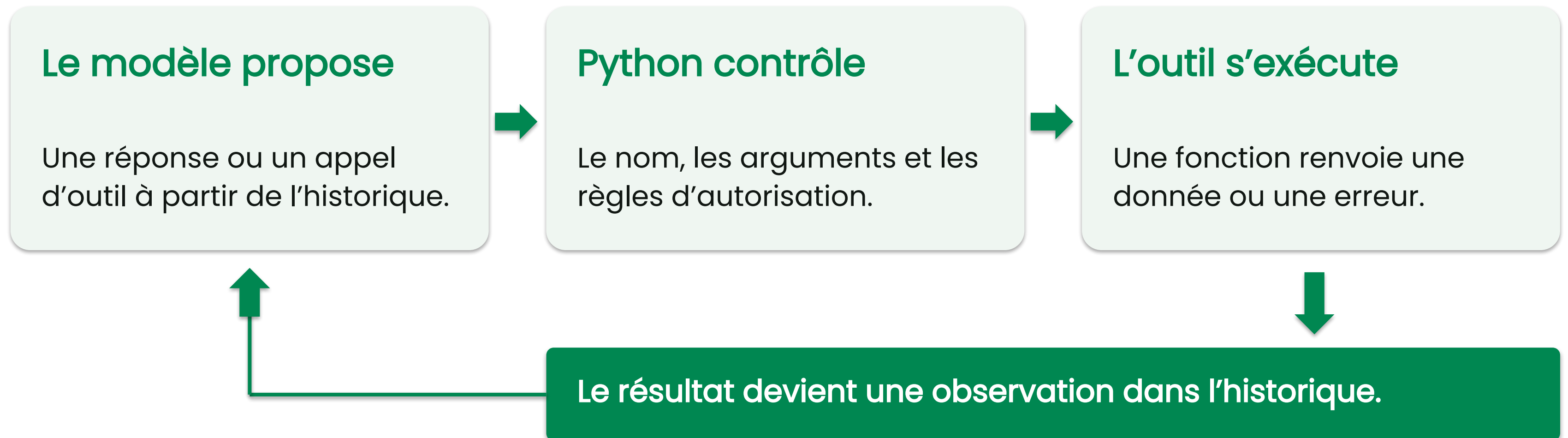
Agent

C'est l'application qui fait alterner le modèle et les outils jusqu'à une réponse ou un arrêt.

Le modèle ne découvre pas les fonctions tout seul : l'application lui transmet leurs descriptions.

Un tour transforme une demande en nouvelle information.

Le résultat d'un outil n'est pas automatiquement la réponse finale.



Un nouveau tour peut commencer.

Lorsque le modèle répond, Python vérifie les preuves avant de rendre la conclusion.

Un outil possède une fonction et un contrat.

La fonction

Elle contient le comportement réel. Ici, elle lit les données locales d'une clinique.

Le schéma

Il décrit au modèle le nom, le but et les arguments autorisés.

Le résultat

Il distingue une réussite avec des données d'un échec explicite.

Le schéma guide le modèle ; la validation et l'exécution restent dans Python.

Valider les arguments avant de lire les données.

```
class ClinicStatusInput(BaseModel):
    clinic_id: str = Field(
        pattern=r"^KCARE-[A-Z]{3}-\d{2}$"
    )

def get_clinic_status(arguments: dict) -> ToolResult:
    values = ClinicStatusInput.model_validate(arguments)
    clinics = json.loads(CLINICS_PATH.read_text())
    clinic = next(
        (row for row in clinics
         if row["clinic_id"] == values.clinic_id),
        None,
    )
    if clinic is None:
        return ToolResult(ok=False, error={
            "type": "ClinicNotFound",
            "message": f"Unknown clinic: {values.clinic_id}",
        })
    return ToolResult(ok=True, data=clinic)
```

CE QUE FAIT CE CODE

- 1 ClinicStatusInput impose le format de l'identifiant.
- 2 La fonction cherche ensuite la clinique dans un fichier local.
- 3 Le résultat annonce clairement succès ou erreur.

Entrée : KCARE-ADJ-01 → Sortie : 12,4 °C · 52 min · capteur OK

Le client masque le fournisseur, pas le comportement attendu.

Une interface stable

Tous les clients exposent `mode` et `complete(messages, tools)`.

Plusieurs implémentations

Gemini appelle une API ;
MockLLM produit un scénario déterministe hors ligne.

Une dépendance explicite

La fonction reçoit `client` en argument. Elle n'a pas besoin de savoir comment il a été créé.

`complete` est une méthode appelée avec des parenthèses ; `mode` est un attribut lu sans parenthèses.

Définir le contrat, puis injecter le client choisi.

```
class LLMClient(Protocol):
    mode: str

    def complete(
        self,
        messages: list[dict],
        tools: list[dict],
    ) -> AssistantTurn: ...

def ask_model(
    state: AgentState,
    client: LLMClient,
) -> AssistantTurn:
    return client.complete(
        state.messages,
        TOOL_SCHEMAS,
    )
```

CE QUE FAIT CE CODE

- 1 LLMClient annonce les membres disponibles.
- 2 client est le paramètre reçu par ask_model.
- 3 TOOL_SCHEMAS présente les cinq outils au modèle.

Même fonction avec Gemini ou MockLLM ;
seul l'objet client change.

Le modèle propose l'appel ; il ne lit pas encore la clinique.

1. AssistantTurn renvoyé

```
{  
  "name": "get_clinic_status",  
  "arguments": {  
    "clinic_id": "KCARE-ADJ-01"  
  }  
}
```

Le modèle choisit un nom connu et prépare ses arguments.

2. Après exécution de l'outil

```
{  
  "ok": true,  
  "data": {  
    "temperature_c": 12.4,  
    "excursion_minutes": 52  
  }  
}
```

Python valide l'appel, exécute la fonction et obtient cette observation.

Représentation normalisée du scénario simulé ; les valeurs viennent du jeu de données local.

L'agent avance grâce à deux mémoires différentes.

Historique

Les messages transmis au modèle : demande, appels proposés et résultats des outils.

Trace

Le journal de contrôle : outil, arguments, statut, résultat, erreur et durée.

Appels déjà vus

Une empreinte des demandes exécutées permet de bloquer une répétition exacte.

Sans observation dans l'historique, le modèle ne sait pas ce que l'outil vient de trouver.

Regrouper ce qui permet de continuer et d'auditer.

```
@dataclass
class AgentState:
    question: str
    messages: list[dict]
    trace: list[TraceEntry] = field(
        default_factory=list
    )
    seen_calls: set[str] = field(
        default_factory=set
    )

def create_state(question: str) -> AgentState:
    return AgentState(
        question=question,
        messages=[
            {"role": "system", "content": SYSTEM_PROMPT},
            {"role": "user", "content": question},
        ],
    )
```

CE QUE FAIT CE CODE

- 1 La question et les messages lancent la conversation.
- 2 La trace est vide au départ.
- 3 seen_calls permettra de reconnaître une répétition.

État initial : 2 messages · 0 outil exécuté · 0 trace

Contrôler, tracer, puis renvoyer l'observation.

```
def execute_and_observe(state, call) -> bool:
    signature = json.dumps({
        "name": call.name,
        "arguments": call.arguments,
    }, sort_keys=True)
    if signature in state.seen_calls:
        raise RuntimeError("Repeated tool call blocked")
    state.seen_calls.add(signature)

    started = time.perf_counter()
    result = execute_checked(
        call, state.trace, state.question
    )
    state.trace.append(
        make_trace(state, call, result, started)
    )
    state.messages.append({
        "role": "tool",
        "tool_call_id": call.id,
        "name": call.name,
        "content": result.model_dump_json(),
    })
    return result.ok
```

CE QUE FAIT CE CODE

- 1 L'empreinte bloque un appel identique déjà exécuté.
- 2 `execute_checked` applique les validations avant l'outil.
- 3 Le résultat va dans la trace et dans l'historique.

Une observation réussie devient le contexte du tour suivant.

Chaque observation détermine l'information encore nécessaire.

Tour	Appel exécuté	Observation utile
1	get_clinic_status	12,4 °C · 52 min · capteur OK
2	search_cold_chain_sop	Seuil de 8 °C pendant 30 min
3	assess_excursion_risk	CRITICAL · revue humaine requise
4	create_incident	Dossier INC-001 créé
5	request_human_review	APPROVED pour l'action proposée

Après ces cinq observations, la conclusion peut être reliée à des preuves.

La boucle décide quand appeler un outil et quand s'arrêter.

Si un appel est proposé

Le programme ajoute la proposition, exécute chaque appel autorisé et renvoie son résultat.

Si une réponse est proposée

Le programme examine les preuves avant de la rendre.

Si rien n'aboutit

Une limite de tours produit un arrêt explicite au lieu d'une boucle infinie.

L'agent n'est pas une fonction magique : c'est cette boucle autour du modèle et des outils.

Assembler les abstractions sans masquer le mécanisme.

```
def run_agent(question, client, max_turns=8):
    state = create_state(question)
    for _ in range(max_turns):
        turn = ask_model(state, client)
        if not turn.tool_calls:
            return finish(
                state, turn.content or "No answer"
            )

        state.messages.append({
            "role": "assistant",
            "content": turn.content,
            "tool_calls": [
                call.model_dump()
                for call in turn.tool_calls
            ],
        })
        for call in turn.tool_calls:
            if not execute_and_observe(state, call):
                return {"status": "blocked"}

    return {"status": "blocked",
            "reason": "turn_limit"}
```

CE QUE FAIT CE CODE

- 1 create_state prépare le contexte.
- 2 ask_model produit chaque nouvelle proposition.
- 3 execute_and_observe relie outil et observation.
- 4 finish contrôle la réponse finale.

La démo exécutable suit cinq appels puis retourne status = approved.

Une sortie structurée peut encore être fausse.

Validation de forme

L'outil existe, les champs sont présents et les types sont acceptables.

Cohérence des preuves

Les valeurs proposées correspondent aux observations réellement obtenues.

Autorisation

Les actions sensibles exigent la décision prévue pour le bon incident.

La sûreté ne tient pas dans le prompt : elle s'appuie aussi sur des contrôles déterministes.

Refuser une mesure qui ne vient pas de l'observation.

```
if call.name == "assess_excursion_risk":
    status = latest(trace, "get_clinic_status")
    sop = latest(trace, "search_cold_chain_sop")
    if not status or not sop:
        return "Telemetry and SOP required"

    expected = {
        "clinic_id": status["clinic_id"],
        "temperature_c": status["temperature_c"],
        "duration_minutes": status[
            "excursion_minutes"
        ],
        "sensor_status": status["sensor_status"],
    }
    if any(call.arguments.get(key) != value
           for key, value in expected.items()):
        return "Arguments differ from telemetry"
```

CE QUE FAIT CE CODE

- 1 Le calcul exige d'abord mesure et procédure.
- 2 expected est construit depuis la trace.
- 3 Chaque argument proposé doit correspondre à cette preuve.

Proposition `temperature_c = 5,0` → appel refusé ; mesure observée = 12,4

Le modèle dit « résolu ». Peut-on accepter sa réponse ?

La réponse proposée

« Tout est sûr ; l'incident est résolu. »

Le modèle vient de lire la mesure, la procédure et le niveau de risque.

Ce que montre la trace

Le risque exige une revue.

Aucun dossier créé.

Aucune approbation enregistrée.

Python bloque la conclusion : `stopped / review_required`. Il manque l'accord requis.

Réponse volontairement fautive du simulateur : « Everything is safe; incident resolved. »

Lier l'accord au bon incident et à la bonne action.

```
approved = bool(
    risk["human_review_required"]
    and incident
    and review
    and review["incident_id"]
        == incident["incident_id"]
    and review["decision"] == "APPROVED"
    and review["approved_action"]
        == risk["recommended_action"]
)

if risk["human_review_required"] and not approved:
    return {"status": "review_required"}
```

CE QUE FAIT CE CODE

- 1 L'accord doit exister et être explicite.
- 2 Il doit viser le dossier de cette exécution.
- 3 L'action approuvée doit être celle recommandée.

APPROVED pour un autre incident ou une autre action → conclusion bloquée

Un test utile vérifie le chemin, les preuves et l'arrêt.

Chemin normal

Les cinq outils sont appelés dans l'ordre et l'accord conforme permet la conclusion.

Décision fautive

Mesure altérée, incident incohérent ou approbation absente doivent être refusés.

Résultat attendu

Un scénario d'erreur passe lorsque le système produit précisément l'arrêt attendu.

10/10 signifie que dix comportements décrits passent ; ce n'est pas une garantie universelle.

Un test passe quand le comportement attendu est respecté.

Scénario	Résultat attendu	Solution	Test
Alerte critique complète	human_approved	human_approved	PASS
Aucune investigation	blocked	blocked	PASS
Approbation absente	review_required	review_required	PASS
Appel identique répété	blocked	blocked	PASS

L'arrêt peut être le résultat correct : les tests vérifient aussi les refus attendus.

Extraits de la solution : dix scénarios déterministes, dont sept erreurs volontaires. Aucun appel API.

Une architecture en cinq responsabilités.

1. Décrire

Présenter au modèle des outils avec un contrat clair.

2. Orchestrer

Alterner propositions, contrôles, exécutions et observations.

3. Prouver

Conserver une trace et vérifier la provenance des arguments.

4. Autoriser

Exiger une décision humaine liée à l'action sensible.

5. Évaluer

Tester les réussites, les erreurs et les refus attendus.

Le code de démonstration reprend ces cinq responsabilités dans des fonctions séparées.

Rejouer la démo, puis explorer l'implémentation complète.

[Démonstration Python guidée](#)

Le code court et exécutable utilisé dans cette présentation.

[Notebook solution français](#)

La mission complète, déjà assemblée, dans Google Colab.

[Notebook participant français](#)

Les quatre décisions à compléter pour pratiquer après la session.

MERCI

Une bonne réponse n'est pas une preuve d'exécution.

Quel outil, quelle preuve et quelle autorisation
votre prochain agent devra-t-il posséder ?

Commencez petit : un outil déterministe, une trace lisible et un cas qui doit être refusé.



[Code et notebooks](#)

[Scannez pour ouvrir le dépôt](#)

BOSSA Chabel · Ingénieur logiciel · ENSGMM

github.com/chabelbossa · linkedin.com/in/chabel-holdo-bossa